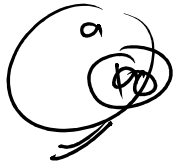
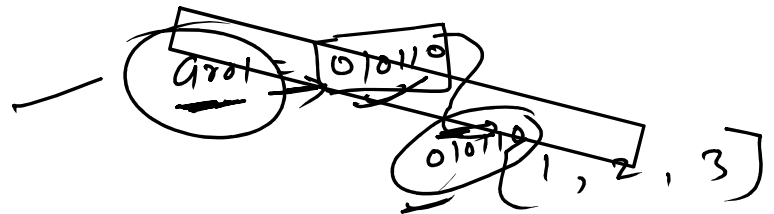
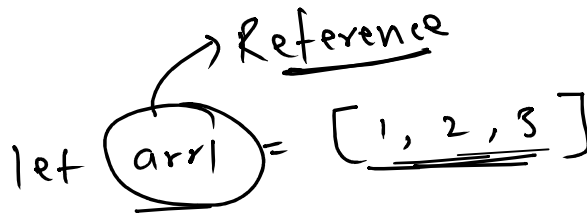


Phase 2: Data Structures



Arrays - Part 1

Non-primitive
or,
Reference



Array

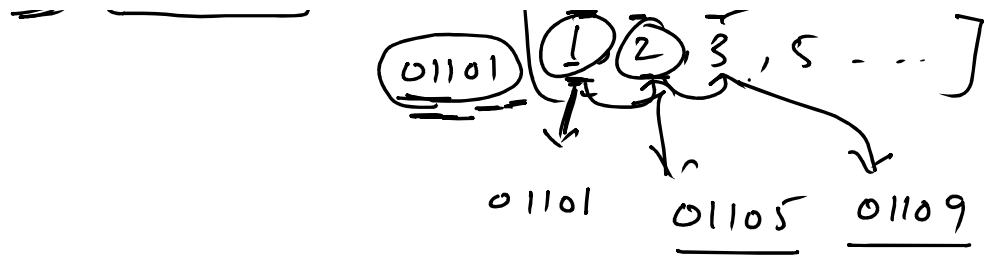
→ An array is used to store multiple values in a variable

→ ordered collection of data ✓

Syntax -

let arr1 = [1, 2, 5, 0, "Satyam"]





Ways to create Array

① Array Literal

— `const array1 = [1, 2, 'Satyam', true]`

② Array Constructor —

`const array1 = new Array(1, 2, 3, 4)`
 ↳ `[1, 2, 3, 4]`

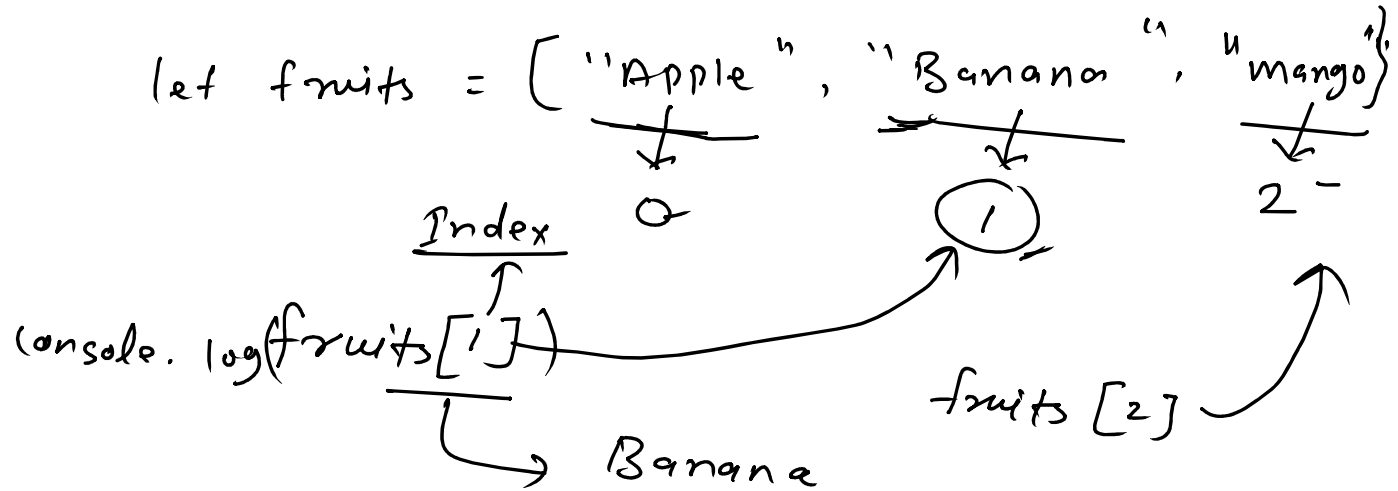
③ Empty Array + Push

`const arr = []`

↳ `arr.push(1) → [1]`
 ↳ `arr.push(5) → [1, 5]`

± Array Index -

→ Array indexing start from 0.



± updating array element

const fruits = ["Apple", "Banana", "Mango"]

→ fruits[1] = "Orange"

(1)

± Array length

→ The length property gives the numbers of elements.

const array1 = [1, 5, 3, 2, 9, 6]

const array1 = [1, 5, 3, 2, 9, 6]

array1.length → 6

last element
[length - 1]

± typeof →

→ typeof array → object ✓

→ so, use

Array.isArray(variable) → True/false

± push() → add at the end ✓

± pop() → last element Remove

± shift() → Remove first element

± unshift() → Add element in first

± Basic Array loop

→ A loop can access every element using its index

```
let fruits = ["Apple", "Mango", "Banana"]
```

```
let length = fruits.length → ③
```

```
for (let i = 0; i < length; i++) {  
    console.log(fruits[i])  
}
```

✓

Slice()

↳ extracts a portion of an array
↳ not change in original.

```
.slice(start, end)  
           ↓           ↘  
        include     end is not  
                    included
```

```
const arr1 = [1, 2, 3, 5]
```

```
arr1.slice(1, 3) ✓
```

Splice() → It is used to add,

Remove or, replace elements.

→ modify the original array.

Syntes

splice (start , deleteCount , items)

- Add element -

↳ use delete count as 0

const arr = ['A', 'B', 'C', 'D']

```
arr.splice(1, 0, 'D')
```

```
console.log(arr)
```

- Replace Element with splice

→ Remove the old value and provide the new value in the same cell.

concat() → Combines arrays and does not modify the original arrays.

```
const arr1 = [1, 2, 3, 4]
```

✓ `const arr1 = [1, 2, 3, 4]`
← `const arr2 = [11, 12, 13, 14]`

`let result = arr1.concat(arr2)`

→ `[1, 2, 3, 4, 11, 12, 13, 14]`

Spread operator →

`[...arr1, ...arr2]` ✓

includes() -

↳ checks whether an array contains a value and return true/false

`let fruits = ['Apple', 'Banana', 'Mango']`

→ `fruits.includes("Banana")`

→ `true`

indexOf()

→ Returns the index of the first

matching element

→ not found → -1

arr.indexof(element)

lastIndexOf(ele)

↳ Returns the index of last occurrence.

Reverse()

↳ reverses an array

↳ modifies the original array